Understand Dataset

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

# import re
# import string

# from collections import Counter
```

```
df=pd.read_csv("/content/Combined Data.csv")
df.head()
```

	Unnamed: 0	statement	status
0	0	oh my gosh	Anxiety
1	1	trouble sleeping, confused mind, restless hear...	Anxiety
2	2	All wrong, back off dear, forward doubt. Stay ...	Anxiety
3	3	I've shifted my focus to something else but I'...	Anxiety
4	4	I'm restless and restless, it's been a month n...	Anxiety

```
df.shape
```

```
(53044, 3)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53044 entries, 0 to 53043
Data columns (total 3 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   Unnamed: 0  53044 non-null  object
 1   statement   52681 non-null  object
 2   status      53042 non-null  object
dtypes: object(3)
memory usage: 1.2+ MB
```

```
df.drop(columns='Unnamed: 0',inplace=True)
```

```
df.isnull().sum()
```

	0
statement	363
status	2

dtype: int64

```
df.dropna(inplace=True)
```

```
df.isnull().sum()
```

	0
statement	0
status	0

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(1592)
```

```
df.drop_duplicates(inplace=True)
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.reset_index(drop=True, inplace=True)
df.shape
```

```
(51088, 2)
```

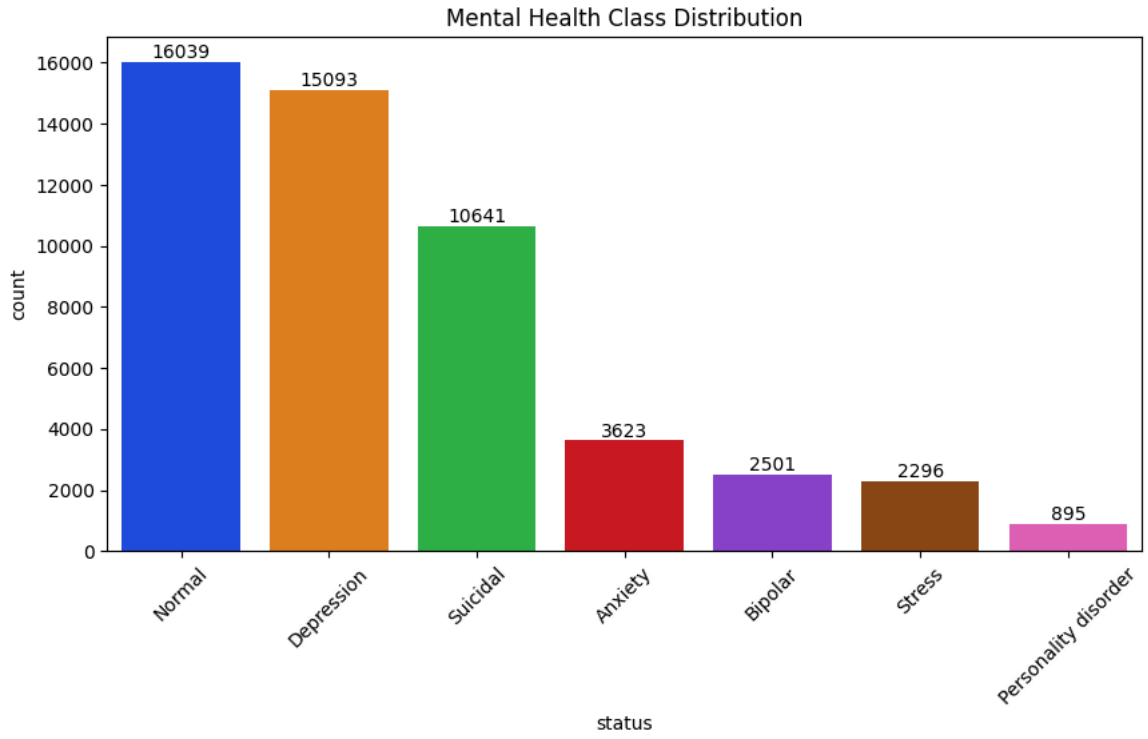
EDA

```
df['status'].value_counts()
```

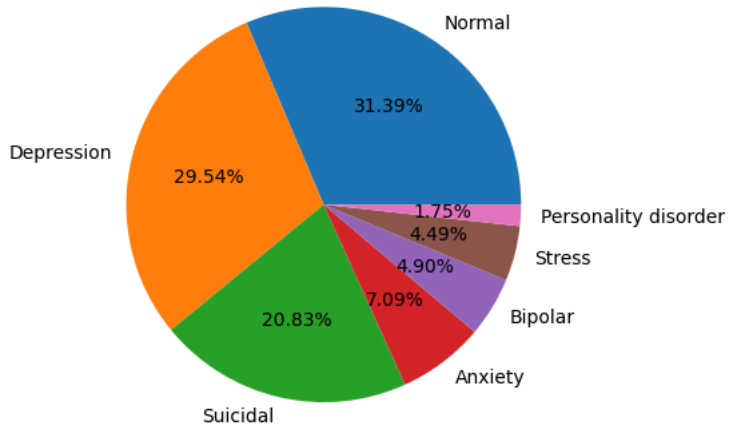
status	count
Normal	16039
Depression	15093
Suicidal	10641
Anxiety	3623
Bipolar	2501
Stress	2296
Personality disorder	895

dtype: int64

```
plt.figure(figsize=(10,5))

ax=sns.countplot(
    data=df,
    x='status',
    order=df['status'].value_counts().index,
    palette="bright"
)
for i in ax.containers:
  ax.bar_label(i)

plt.title("Mental Health Class Distribution")
plt.xticks(rotation=45)
plt.show()
```




```
counts = df['status'].value_counts()
labels = counts.index

plt.pie(counts, labels=labels, autopct='%1.2f%%')
plt.show()
```




```
df['sentence_length'] = df['statement'].apply(len)
df.head()
```

	statement	status	sentence_length
0	oh my gosh	Anxiety	10
1	trouble sleeping, confused mind, restless hear...	Anxiety	64
2	All wrong, back off dear, forward doubt. Stay ...	Anxiety	78
3	I've shifted my focus to something else but I'...	Anxiety	61
4	I'm restless and restless, it's been a month n...	Anxiety	72

```
# df['sentence_length'].describe()
```

```
# plt.figure(figsize=(10,5))

# sns.histplot(df['char_length'], bins=50)

# plt.title("Character Length Distribution")
```

```
# plt.show()
```

```
# plt.figure(figsize=(25, 6))  # Slightly wider figure so the numbers don't overlap

# # Define the sequence from 5 up to 100 (inclusive of 100) with a step of 5
# custom_bins = list(range(5, 1000, 5))

# # Pass the sequence to the bins parameter
# sns.histplot(df["word_count"], bins=custom_bins, kde=True)

# # Zoom in on the 0 to 105 range so the last bin fits nicely
# plt.xlim(0, 105)

# # Set the x-axis tick marks exactly at your bin intervals
# plt.xticks(list(range(5, 350, 5)))

# plt.title("Distribution of Statement Length")
# plt.xlabel("Number of Words")
# plt.show()
```

```
df['word_count'] = df['statement'].apply(lambda x: len(str(x).split()))
df.head()
```

Show hidden output

```
# df['word_count'].describe()
```

```
# plt.figure(figsize=(10,5))

# sns.histplot(df['word_count'], bins=50)

# plt.title("Word Count Distribution")

# plt.show()
```

```
# df['avg_word_length'] = (df['sentence_length'] / df['word_count'])
# df.head()
```

```
df['vocabulary_size'] = df['statement'].apply(lambda x: len(set(str(x).split())))
df.head()
```

Show hidden output

```
from collections import Counter

all_words = " ".join(df['statement']).split()

word_freq = Counter(all_words)

word_freq.most_common(20)
```

```
[('I', 309231),
 ('to', 187663),
 ('and', 162747),
 ('the', 115333),
 ('a', 111401),
 ('my', 105434),
 ('of', 80997),
 ('i', 77820),
 ('not', 71591),
 ('is', 66935),
 ('have', 66845),
 ('it', 63620),
 ('that', 61135),
 ('am', 59957),
 ('in', 56760),
 ('me', 54410),
 ('do', 50218),
 ('for', 50178),
 ('but', 47067),
 ('just', 45084)]
```

```
# from wordcloud import WordCloud

# text = " ".join(df['statement'])

# wordcloud = WordCloud(
#     width=1200,
```

```
#     height=600,
#     background_color='white'
# ).generate(text)

# plt.figure(figsize=(15,8))

# plt.imshow(wordcloud)

# plt.axis("off")

# plt.show()
```

Preprocessing

```
df['statement'].sample(5,random_state=10).values
```

Show hidden output

```
# Install required libraries
!pip install emoji

# Standard Libraries
import re
import html
import unicodedata

# Third-party Libraries
import emoji
```

Show hidden output

```
def normalize_unicode(text):
    """Normalize Unicode characters."""
    return unicodedata.normalize("NFKC", text)


def remove_urls(text):
    """Remove URLs."""
    return re.sub(r"http\S+|www\S+", "", text)


def remove_html(text):
    """Remove HTML tags and decode HTML entities."""
    text = html.unescape(text)
    return re.sub(r"<.*?>", "", text)


def remove_emails(text):
    """Remove email addresses."""
    return re.sub(r"\S+@\S+", "", text)


def remove_mentions(text):
    """Remove social media mentions."""
    return re.sub(r"@\w+", "", text)


def convert_emojis(text):
    """Convert emojis into text descriptions."""
    return emoji.demojize(text, delimiters=(" ", " "))


def remove_extra_spaces(text):
    """Normalize multiple spaces."""
    return re.sub(r"\s+", " ", text).strip()
```

```
def preprocess_text(text):
    """
    Minimal preprocessing for DistilBERT.
    Preserve natural language as much as possible.
    """

    text = str(text)

    text = normalize_unicode(text)
    text = remove_html(text)
    text = remove_urls(text)
```

```
    text = remove_emails(text)
    text = remove_mentions(text)
    text = convert_emojis(text)
    text = remove_extra_spaces(text)

    return text
```

```
df["bert_text"] = df["statement"].apply(preprocess_text)
```

```
comparison = pd.DataFrame({
    "Original": df["statement"],
    "BERT Text": df["bert_text"]
})

comparison.head(10)
```

Show hidden output

```
df['bert_text'].sample(5,random_state=10).values
```

Show hidden output

Label Encoding

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()

df["label"] = label_encoder.fit_transform(df["status"])

num_classes = len(label_encoder.classes_)

print(label_encoder.classes_)
print(f"Number of classes: {num_classes}")

# print(y[49200:49300])
```

```
['Anxiety' 'Bipolar' 'Depression' 'Normal' 'Personality disorder' 'Stress'
 'Suicidal']
Number of classes: 7
```

```
label_mapping = dict(zip(label_encoder.classes_,
                         label_encoder.transform(label_encoder.classes_)))

print(label_mapping)
```

```
{'Anxiety': np.int64(0), 'Bipolar': np.int64(1), 'Depression': np.int64(2), 'Normal': np.int64(3), 'Personality disorder': r
```

Train-Test Split

```
from sklearn.model_selection import train_test_split

X = df["bert_text"]
y = df["label"]

# 80% Train, 20% Temporary
X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Split the remaining 20% into Validation (10%) and Test (10%)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=42,
    stratify=y_temp
)
```

```
print(f"Training Samples   : {len(X_train)}")
print(f"Validation Samples : {len(X_val)}")
print(f"Testing Samples    : {len(X_test)}")
```

```
Training Samples   : 40870
Validation Samples : 5109
Testing Samples    : 5109
```

```
print("Train Distribution")
print(y_train.value_counts(normalize=True).sort_index())

print("\nValidation Distribution")
print(y_val.value_counts(normalize=True).sort_index())

print("\nTest Distribution")
print(y_test.value_counts(normalize=True).sort_index())
```

```
Train Distribution
label
0    0.070908
1    0.048960
2    0.295425
3    0.313947
4    0.017519
5    0.044947
6    0.208295
Name: proportion, dtype: float64

Validation Distribution
label
0    0.071051
1    0.048933
2    0.295557
3    0.313956
4    0.017420
5    0.044823
6    0.208260
Name: proportion, dtype: float64

Test Distribution
label
0    0.070855
1    0.048933
2    0.295361
3    0.313956
4    0.017616
5    0.045019
6    0.208260
Name: proportion, dtype: float64
```

Load DistilBERT Tokenizer

```
# Sentence
#       ↓
# DistilBERT Tokenizer
#       ↓
# Input IDs
#       ↓
# Pretrained DistilBERT Embeddings
```

```
!pip install -q transformers datasets accelerate tqdm
```

```
from transformers import AutoTokenizer
```

```
MODEL_NAME = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
config.json:  100%                                                  483/483  [00:00<00:00,  5.85kB/s]
tokenizer_config.json:  100%                                                  48.0/48.0  [00:00<00:00,  862B/s]
vocab.txt:  100%                                                  232k/232k  [00:00<00:00,  3.23MB/s]
tokenizer.json:  100%                                                  466k/466k  [00:00<00:00,  3.97MB/s]
```

```
print("Vocabulary Size :", tokenizer.vocab_size)
print("Model Max Length:", tokenizer.model_max_length)
```

```
print("Padding Token   :", tokenizer.pad_token)
print("CLS Token       :", tokenizer.cls_token)
print("SEP Token       :", tokenizer.sep_token)
print("UNK Token       :", tokenizer.unk_token)
```

Show hidden output

```
sample_text = X_train.iloc[0]

print(sample_text)
```

```
a pc or a mac?
```

```
encoded = tokenizer(sample_text)

encoded
```

```
{'input_ids': [101, 1037, 7473, 2030, 1037, 6097, 1029, 102], 'token_type_ids': [0, 0, 0, 0, 0, 0, 0, 0], 'attention_mask':
[1, 1, 1, 1, 1, 1, 1, 1]}
```

```
print("Input IDs:")
print(encoded["input_ids"])

print("\nAttention Mask:")
print(encoded["attention_mask"])
```

```
Input IDs:
[101, 1037, 7473, 2030, 1037, 6097, 1029, 102]

Attention Mask:
[1, 1, 1, 1, 1, 1, 1, 1]
```

```
tokens = tokenizer.convert_ids_to_tokens(encoded["input_ids"])

print(tokens)
```

```
['[CLS]', 'a', 'pc', 'or', 'a', 'mac', '?', '[SEP]']
```

```
for token, token_id in zip(tokens, encoded["input_ids"]):
    print(f"{token:<15} -> {token_id}")
```

```
[CLS]           -> 101
a               -> 1037
pc              -> 7473
or              -> 2030
a               -> 1037
mac             -> 6097
?               -> 1029
[SEP]           -> 102
```

```
sample = "I am feeling unbelievably depressed today."

encoded = tokenizer(sample)

tokens = tokenizer.convert_ids_to_tokens(encoded["input_ids"])

print(tokens)
```

```
['[CLS]', 'i', 'am', 'feeling', 'un', '##bel', '##ie', '##va', '##bly', 'depressed', 'today', '.', '[SEP]']
```

Finding the Optimal max_length

```
token_lengths = X_train.apply(
    lambda x: len(tokenizer.encode(x, add_special_tokens=True))
)
```

```
print(token_lengths.describe())
```

```
count    40870.000000
mean       134.997871
std        188.922508
min          2.000000
25%         22.000000
50%         75.000000
75%        175.000000
max       5857.000000
Name: bert_text, dtype: float64
```

```
print("90th percentile :", token_lengths.quantile(0.90))
print("95th percentile :", token_lengths.quantile(0.95))
print("99th percentile :", token_lengths.quantile(0.99))
print("Maximum :", token_lengths.max())
```

```
90th percentile : 328.0
95th percentile : 462.0
99th percentile : 862.3099999999977
Maximum : 5857
```

```
# import matplotlib.pyplot as plt

# plt.figure(figsize=(10, 5))
# plt.hist(token_lengths, bins=50)
# plt.xlabel("Token Length")
# plt.ylabel("Number of Samples")
# plt.title("Distribution of Token Lengths")
# plt.show()
```

Tokenize the Entire Dataset

```
MAX_LENGTH = 384 #512

train_encodings = tokenizer(
    X_train.tolist(),
    truncation=True,
    padding=True,
    max_length=MAX_LENGTH
)

val_encodings = tokenizer(
    X_val.tolist(),
    truncation=True,
    padding=True,
    max_length=MAX_LENGTH
)

test_encodings = tokenizer(
    X_test.tolist(),
    truncation=True,
    padding=True,
    max_length=MAX_LENGTH
)
```

Creating a Custom PyTorch Dataset

```
import torch
from torch.utils.data import Dataset
```

```
class MentalHealthDataset(Dataset):

    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels.tolist()

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):

        item = {
            key: torch.tensor(val[idx])
            for key, val in self.encodings.items()
        }

        item["labels"] = torch.tensor(self.labels[idx])

        return item
```

```
train_dataset = MentalHealthDataset(
    train_encodings,
    y_train
)

val_dataset = MentalHealthDataset(
```

```
    val_encodings,
    y_val
)

test_dataset = MentalHealthDataset(
    test_encodings,
    y_test
)
```

```
sample = train_dataset[0]

print(sample.keys())
```

```
dict_keys(['input_ids', 'token_type_ids', 'attention_mask', 'labels'])
```

```
print(sample["input_ids"].shape)
print(sample["attention_mask"].shape)
print(sample["labels"])
```

```
torch.Size([384])
torch.Size([384])
tensor(3)
```

```
print(sample["input_ids"].dtype)
print(sample["attention_mask"].dtype)
print(sample["labels"].dtype)
```

```
torch.int64
torch.int64
torch.int64
```

DataLoader

```
from torch.utils.data import DataLoader
```

```
BATCH_SIZE = 16

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False
)
```

```
batch = next(iter(train_loader))
```

```
print(batch["input_ids"].shape)
print(batch["attention_mask"].shape)
print(batch["labels"].shape)
```

```
torch.Size([16, 384])
torch.Size([16, 384])
torch.Size([16])
```

Loading the Pretrained Model

```
# Embedding Layer
#         ↓
# BiLSTM
#         ↓
# Attention
#         ↓
# Dense
```

```
#         ↓
# Softmax
```

```
# DistilBERT Encoder
#         ↓
# Transformer Layer 1
#         ↓
# Transformer Layer 2
#         ↓
# Transformer Layer 3
#         ↓
# Transformer Layer 4
#         ↓
# Transformer Layer 5
#         ↓
# Transformer Layer 6
```

```
# Input IDs
#       ↓
# Embedding Layer
#       ↓
# Transformer Block × 6
#       ↓
# CLS Representation
#       ↓
# Dropout
#       ↓
# Linear Classifier
#       ↓
# 7 Class Probabilities
```

```
from transformers import DistilBertForSequenceClassification
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = DistilBertForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=num_classes
)
model.to(device)
```

```
model.safetensors: reconstructing file: 100%                    268MB / 268MB, 24.8MB/s

model.safetensors: downloading bytes:                           250MB, 22.4MB/s

Loading weights: 100%                                           100/100 [00:00<00:00, 3526.91it/s]
[transformers] DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                     | Status     |
------------------------+------------+-
vocab_transform.bias    | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_layer_norm.bias   | UNEXPECTED |
vocab_projector.bias    | UNEXPECTED |
vocab_transform.weight  | UNEXPECTED |
pre_classifier.weight   | MISSING    |
classifier.bias         | MISSING    |
classifier.weight       | MISSING    |
pre_classifier.bias     | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING:  those params were newly initialized because missing from the checkpoint. Consider training on your downstream ta
DistilBertForSequenceClassification(
  (distilbert): DistilBertModel(
    (embeddings): Embeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (transformer): Transformer(
      (layer): ModuleList(
        (0-5): 6 x TransformerBlock(
          (attention): DistilBertSelfAttention(
            (q_lin): Linear(in_features=768, out_features=768, bias=True)
            (k_lin): Linear(in_features=768, out_features=768, bias=True)
            (v_lin): Linear(in_features=768, out_features=768, bias=True)
            (out_lin): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (sa_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (ffn): FFN(
            (dropout): Dropout(p=0.1, inplace=False)
            (lin1): Linear(in_features=768, out_features=3072, bias=True)
            (lin2): Linear(in_features=3072, out_features=768, bias=True)
            (activation): GELUActivation()
          )
          (output_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        )
      )
    )
  )
  (pre_classifier): Linear(in_features=768, out_features=768, bias=True)
  (classifier): Linear(in_features=768, out_features=7, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)
```

```
print(model)
```

Show hidden output

```
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f"Total Parameters: {total_params:,}")
print(f"Trainable Parameters: {trainable_params:,}")
```

```
Total Parameters: 66,958,855
Trainable Parameters: 66,958,855
```

The Forward Pass

```
batch = next(iter(train_loader))
```

```
input_ids = batch["input_ids"].to(device)
attention_mask = batch["attention_mask"].to(device)
labels = batch["labels"].to(device)

outputs = model(
    input_ids=input_ids,
    attention_mask=attention_mask,
    labels=labels
)
```

```
import torch

print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else "No GPU")
```

```
True
Tesla T4
```

```
print(type(outputs))
print(outputs.keys())
```

```
<class 'transformers.modeling_outputs.SequenceClassifierOutput'>
odict_keys(['loss', 'logits'])
```

```
print("Loss:", outputs.loss)

print("Logits Shape:", outputs.logits.shape)

print(outputs.logits)
```

Show hidden output

```
import torch

probabilities = torch.softmax(outputs.logits, dim=1)

print(probabilities.shape)
print(probabilities[0])
print(probabilities[0].sum())
```

```
torch.Size([16, 7])
tensor([0.1450, 0.1419, 0.1247, 0.1256, 0.1645, 0.1438, 0.1545],
       device='cuda:0', grad_fn=<SelectBackward0>)
tensor(1., device='cuda:0', grad_fn=<SumBackward0>)
```

```
predictions = torch.argmax(outputs.logits, dim=1)

print(predictions)
```

```
tensor([4, 6, 4, 6, 6, 4, 6, 4, 4, 6, 4, 6, 6, 6, 6, 4], device='cuda:0')
```

Optimizer & Learning Rate Scheduler

```
# Forward Pass
#       ↓
# Compute Loss
#       ↓
# Backpropagation
#       ↓
# Optimizer updates weights
```

```
from torch.optim import AdamW
from transformers import get_linear_schedule_with_warmup
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    classification_report,
    confusion_matrix,
    roc_auc_score
)

optimizer = AdamW(
    model.parameters(),
    lr=2e-5,
    weight_decay=0.01
)

EPOCHS = 3 #5 #3
```

```
total_training_steps = len(train_loader) * EPOCHS

print("Training Steps:", total_training_steps)

scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0,
    num_training_steps=total_training_steps
)
```

```
Training Steps: 7665
```

Training Loop

```
# For every epoch
#     For every batch
#         1. Move data to device
#         2. Forward Pass
#         3. Compute Loss
#         4. Backpropagation
#         5. Optimizer Step
#         6. Scheduler Step
#         7. Record Metrics
```

```
# ============================
# Training History
# ============================

train_losses = []
train_accuracies = []

val_losses = []
val_accuracies = []

val_precisions = []
val_recalls = []
val_f1_scores = []
```

```
def train_one_epoch(
        model,
        dataloader,
        optimizer,
        scheduler,
        device
):
    model.train()

    running_loss = 0
    correct = 0
    total = 0

    progress_bar = tqdm(
        dataloader,
        desc="Training"
    )

    for batch in progress_bar:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        optimizer.zero_grad()

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

        loss = outputs.loss

        predictions = torch.argmax(
            outputs.logits,
            dim=1
        )

        correct += (predictions == labels).sum().item()
        total += labels.size(0)
```

```
        loss.backward()

        optimizer.step()
        scheduler.step()

        running_loss += loss.item()

        progress_bar.set_postfix(
            loss=f"{loss.item():.4f}",
            acc=f"{100 * correct / total:.2f}%"
        )

    epoch_loss = running_loss / len(dataloader)
    epoch_accuracy = 100 * correct / total

    # Removed history appends from here, handled in main loop

    return epoch_loss, epoch_accuracy
```

Validation loop

```
def validate_one_epoch(
        model,
        dataloader,
        device
):
    model.eval()

    running_loss = 0
    correct = 0
    total = 0

    all_predictions = []
    all_labels = []

    with torch.no_grad():
        for batch in dataloader:
            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            labels = batch["labels"].to(device)

            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels
            )

            loss = outputs.loss

            predictions = torch.argmax(
                outputs.logits,
                dim=1
            )

            correct += (predictions == labels).sum().item()
            total += labels.size(0)

            running_loss += loss.item()

            all_predictions.extend(
                predictions.cpu().numpy()
            )

            all_labels.extend(
                labels.cpu().numpy()
            )

    # Step 10 — Compute Epoch Metrics
    epoch_loss = running_loss / len(dataloader)
    epoch_accuracy = 100 * correct / total

    # Step 11 — Compute Precision, Recall and F1
    precision, recall, f1, _ = precision_recall_fscore_support(
        all_labels,
        all_predictions,
        average="macro",
        zero_division=0
    )
```

```
    # Removed: Step 12 — Save Validation History

    # Step 13 — Return the Results
    return (
        epoch_loss,
        epoch_accuracy,
        precision,
        recall,
        f1
    )
```

Fit

```
# ============================
# Best Model Tracking
# ============================

best_val_accuracy = 0

best_epoch = 0

patience = 3 # Increased patience as suggested
patience_counter = 0
```

```
import time

start_time = time.time() # Added time tracking

for epoch in range(EPOCHS):

    print("=" * 70)

    print(f"Epoch {epoch+1}/{EPOCHS}")

    print("=" * 70)

    # Step 3 — Train
    train_loss, train_accuracy = train_one_epoch(
        model,
        train_loader,
        optimizer,
        scheduler,
        device
    )

    # Step 4 — Validate
    (
        val_loss,
        val_accuracy,
        val_precision,
        val_recall,
        val_f1
    ) = validate_one_epoch(
        model,
        val_loader,
        device
    )

    # Append to history lists in the main loop
    train_losses.append(train_loss)
    train_accuracies.append(train_accuracy)

    val_losses.append(val_loss)
    val_accuracies.append(val_accuracy)
    val_precisions.append(val_precision)
    val_recalls.append(val_recall)
    val_f1_scores.append(val_f1)

    # Step 5 — Print Metrics
    print(f"\nTraining Loss      : {train_loss:.4f}")
    print(f"Training Accuracy  : {train_accuracy:.2f}%")

    print(f"\nValidation Loss    : {val_loss:.4f}")
    print(f"Validation Accuracy: {val_accuracy:.2f}%")
    print(f"Validation Precision: {val_precision:.4f}")
    print(f"Validation Recall   : {val_recall:.4f}")
    print(f"Validation F1-Score : {val_f1:.4f}")
```

```
    # Step 6 — Save the Best Model
    if val_accuracy > best_val_accuracy:

        best_val_accuracy = val_accuracy

        best_epoch = epoch + 1

        patience_counter = 0

        torch.save(
            {
                "epoch": epoch,
                "model_state_dict": model.state_dict(),
                "optimizer_state_dict": optimizer.state_dict(),
                "best_val_accuracy": best_val_accuracy,
            },
            "best_distilbert.pt"
        )

        print("✅ Best model saved!")

    else:

        patience_counter += 1

    # Step 7 — Early Stopping
    if patience_counter >= patience:

        print("\nEarly stopping triggered!")

        break

end_time = time.time() # Added time tracking

print(f"\nTotal Training Time : {(end_time - start_time)/60:.2f} minutes") # Print total training time
```

```
======================================================================
Epoch 1/3
======================================================================
Training: 100%                                          2555/2555 [24:31<00:00,  2.12it/s, acc=80.04%, loss=0.0227]

Training Loss      : 0.5011
Training Accuracy  : 80.04%

Validation Loss    : 0.4349
Validation Accuracy: 82.50%
Validation Precision: 0.8136
Validation Recall   : 0.7615
Validation F1-Score : 0.7767
✅ Best model saved!
======================================================================
Epoch 2/3
======================================================================
Training: 100%                                          2555/2555 [24:30<00:00,  2.11it/s, acc=85.69%, loss=0.2134]

Training Loss      : 0.3520
Training Accuracy  : 85.69%

Validation Loss    : 0.3863
Validation Accuracy: 84.36%
Validation Precision: 0.8152
Validation Recall   : 0.8114
Validation F1-Score : 0.8126
✅ Best model saved!
======================================================================
Epoch 3/3
======================================================================
Training: 100%                                          2555/2555 [24:30<00:00,  2.12it/s, acc=89.46%, loss=0.3205]

Training Loss      : 0.2675
Training Accuracy  : 89.46%

Validation Loss    : 0.4001
Validation Accuracy: 84.46%
Validation Precision: 0.8178
Validation Recall   : 0.8057
Validation F1-Score : 0.8108
✅ Best model saved!

Total Training Time : 77.55 minutes
```

```
# Step 8 — Final Summary
print("=" * 70)
```

```
print(f"Best Validation Accuracy : {best_val_accuracy:.2f}%")

print(f"Best Epoch               : {best_epoch}") # Adjusted spacing
print(f"Model Saved As           : best_distilbert.pt") # Added model save name

print("=" * 70)
```

```
======================================================================
Best Validation Accuracy : 84.46%
Best Epoch               : 3
Model Saved As           : best_distilbert.pt
======================================================================
```

Test Evaluation

```
model.load_state_dict(torch.load("best_distilbert.pt")["model_state_dict"])
```

```
<All keys matched successfully>
```

```
model.eval()

test_predictions = []
test_labels = []

with torch.no_grad():
    for batch in test_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

        predictions = torch.argmax(
            outputs.logits,
            dim=1
        )

        test_predictions.extend(
            predictions.cpu().numpy()
        )

        test_labels.extend(
            labels.cpu().numpy()
        )
```

```
test_accuracy = accuracy_score(test_labels, test_predictions)
test_precision, test_recall, test_f1, _ = precision_recall_fscore_support(test_labels, test_predictions, average='macro', z

print(f"Test Accuracy   : {test_accuracy:.4f}")
print(f"Test Precision  : {test_precision:.4f}")
print(f"Test Recall     : {test_recall:.4f}")
print(f"Test F1-Score   : {test_f1:.4f}")
```

```
Test Accuracy   : 0.8281
Test Precision  : 0.8045
Test Recall     : 0.7895
Test F1-Score   : 0.7951
```

Classification Report

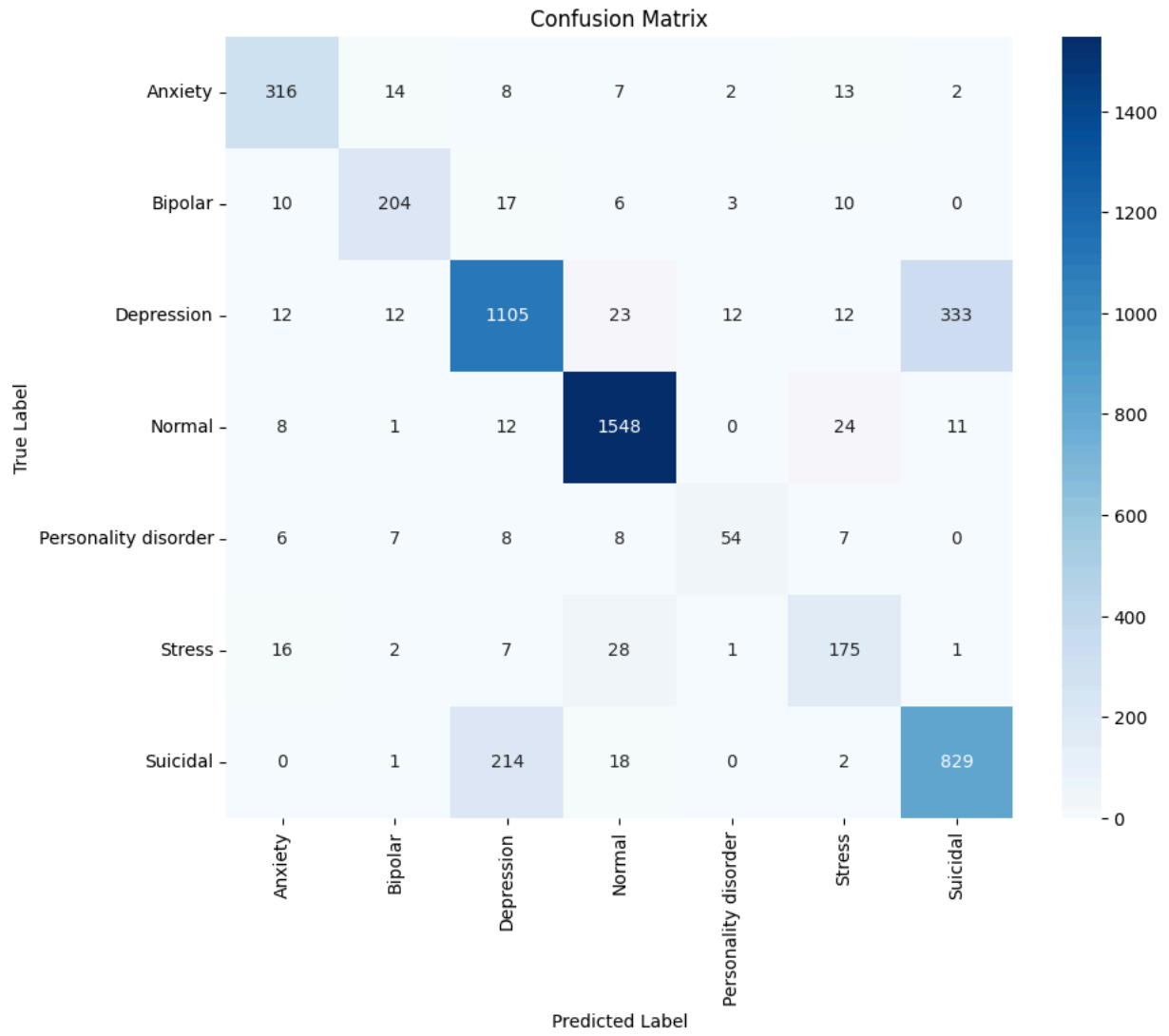
```
print(classification_report(test_labels, test_predictions, target_names=label_encoder.classes_))
```

```
                     precision    recall  f1-score   support

            Anxiety       0.86      0.87      0.87       362
            Bipolar       0.85      0.82      0.83       250
         Depression       0.81      0.73      0.77      1509
             Normal       0.95      0.97      0.95      1604
Personality disorder       0.75      0.60      0.67        90
             Stress       0.72      0.76      0.74       230
           Suicidal       0.70      0.78      0.74      1064

           accuracy                           0.83      5109
          macro avg       0.80      0.79      0.80      5109
```

```
weighted avg       0.83      0.83      0.83      5109
```

Confusion Matrix

```
cm = confusion_matrix(test_labels, test_predictions)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



ROC-AUC

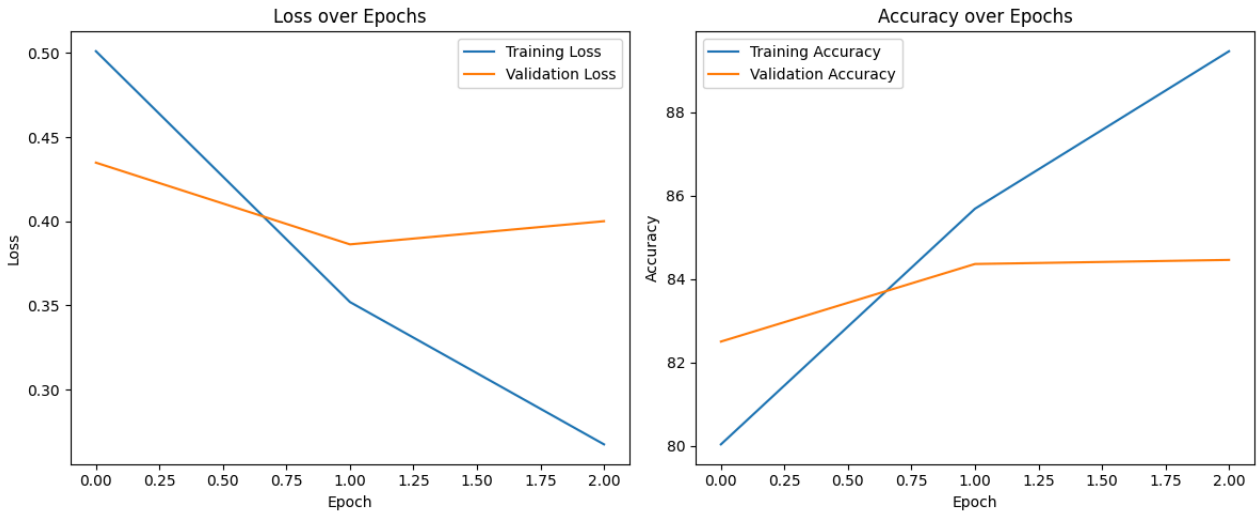
```
from sklearn.preprocessing import LabelBinarizer

# Binarize the labels for ROC AUC calculation
label_binarizer = LabelBinarizer()
y_test_binarized = label_binarizer.fit_transform(test_labels)

# Get predicted probabilities for ROC AUC
model.eval()
all_probs = []
with torch.no_grad():
    for batch in test_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask
        )
        probabilities = torch.softmax(outputs.logits, dim=1)
        all_probs.extend(probabilities.cpu().numpy())
```

```
roc_auc = roc_auc_score(y_test_binarized, all_probs, average='macro')
print(f"Macro-averaged ROC AUC: {roc_auc:.4f}")
```

```
Macro-averaged ROC AUC: 0.9768
```

Learning Curves

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(train_losses, label='Training Loss')
plt.plot(val_losses, label='Validation Loss')
plt.title('Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(train_accuracies, label='Training Accuracy')
plt.plot(val_accuracies, label='Validation Accuracy')
plt.title('Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()
```



Performance on Unseen dataset

```
unseen_df = pd.read_excel('/content/updated_70_samples.xlsx')     #df.sample(100)#pd.read_csv('/content/synthetic_unseen_da
unseen_df.head()
```

Show hidden output

```
all_probs = []

model.eval()

with torch.no_grad():
    for batch in unseen_loader:

        outputs = model(
            input_ids=batch["input_ids"].to(device),
            attention_mask=batch["attention_mask"].to(device)
        )

        probs = torch.softmax(outputs.logits, dim=1)

        all_probs.append(probs.cpu())
```

```
all_probs = torch.cat(all_probs)

print(all_probs.mean(dim=0))
```

Show hidden output

```
# unseen_df.drop(columns='Unnamed: 0')
```

```
unseen_df["bert_text"] = unseen_df["rewritten"].apply(preprocess_text)
# Correct the specific casing issue for 'Personality Disorder'
unseen_df['true_label'] = unseen_df['status'].replace('Personality Disorder', 'Personality disorder')

# Drop rows with NaN values in 'true_label' before encoding
unseen_df.dropna(subset=['true_label'], inplace=True)

unseen_df["label"] = label_encoder.transform(unseen_df["true_label"])

unseen_encodings = tokenizer(
    unseen_df["bert_text"].tolist(),
    truncation=True,
    padding=True,
    max_length=MAX_LENGTH
)

unseen_dataset = MentalHealthDataset(
    unseen_encodings,
    unseen_df["label"]
)

unseen_loader = DataLoader(
    unseen_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False
)
```

```
model.eval()

unseen_predictions = []
unseen_labels = []

with torch.no_grad():
    for batch in unseen_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

        predictions = torch.argmax(
            outputs.logits,
            dim=1
        )

        unseen_predictions.extend(
            predictions.cpu().numpy()
        )

        unseen_labels.extend(
            labels.cpu().numpy()
        )
```

```
unseen_accuracy = accuracy_score(unseen_labels, unseen_predictions)
unseen_precision, unseen_recall, unseen_f1, _ = precision_recall_fscore_support(unseen_labels, unseen_predictions, average=

print(f"Unseen Accuracy   : {unseen_accuracy:.4f}")
print(f"Unseen Precision  : {unseen_precision:.4f}")
print(f"Unseen Recall     : {unseen_recall:.4f}")
print(f"Unseen F1-Score   : {unseen_f1:.4f}")
```

```
Unseen Accuracy   : 0.8143
Unseen Precision  : 0.8518
Unseen Recall     : 0.8143
Unseen F1-Score   : 0.8186
```

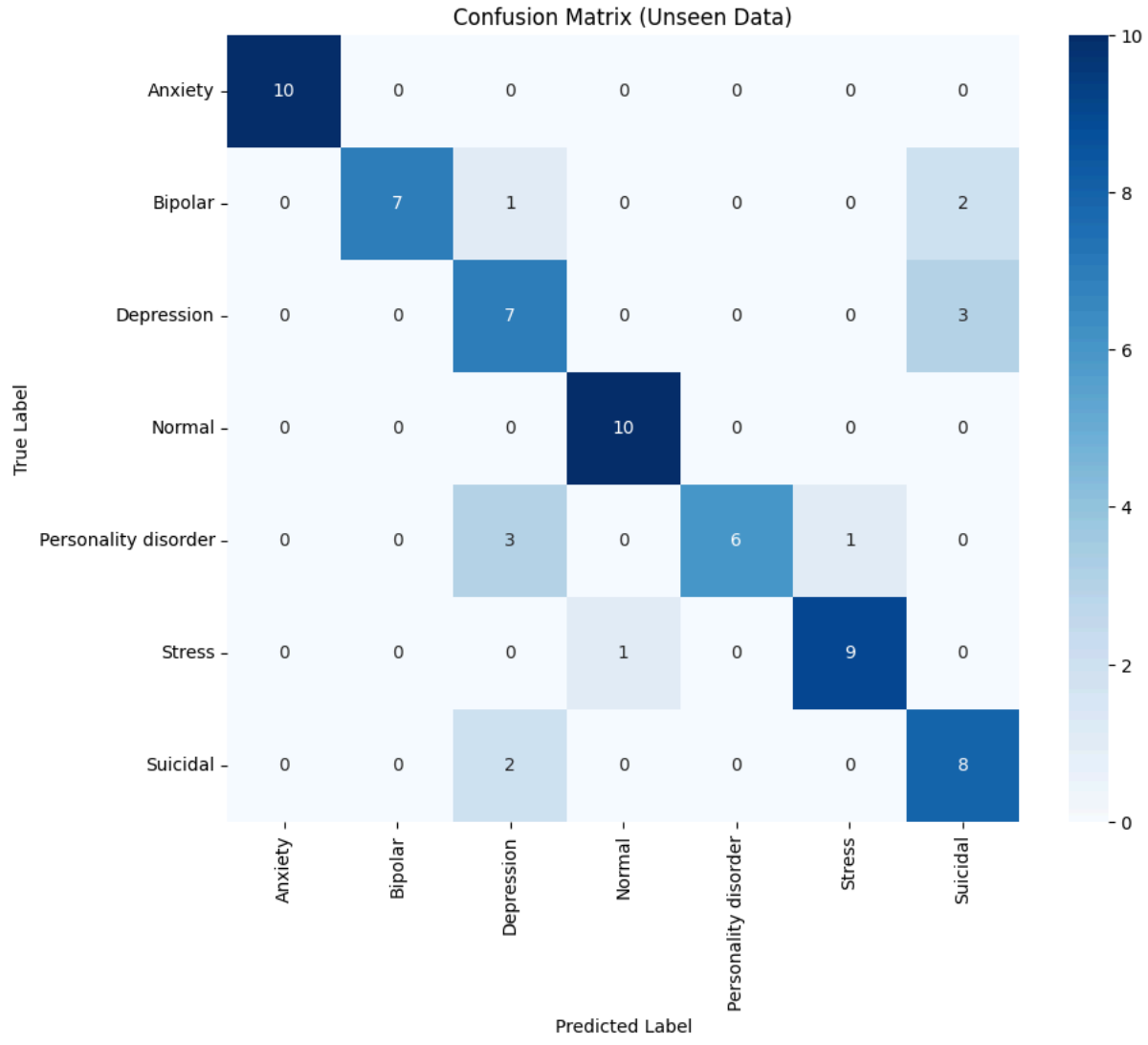
Classification Report (Unseen Data)

```
print(classification_report(unseen_labels, unseen_predictions, target_names=label_encoder.classes_))
```

```
                     precision    recall  f1-score   support

            Anxiety       1.00      1.00      1.00        10
            Bipolar       1.00      0.70      0.82        10
         Depression       0.54      0.70      0.61        10
             Normal       0.91      1.00      0.95        10
Personality disorder       1.00      0.60      0.75        10
             Stress       0.90      0.90      0.90        10
           Suicidal       0.62      0.80      0.70        10

           accuracy                           0.81        70
          macro avg       0.85      0.81      0.82        70
       weighted avg       0.85      0.81      0.82        70
```

Confusion Matrix (Unseen Data)

```
cm_unseen = confusion_matrix(unseen_labels, unseen_predictions)
plt.figure(figsize=(10, 8))
sns.heatmap(cm_unseen, annot=True, fmt='d', cmap='Blues', xticklabels=label_encoder.classes_, yticklabels=label_encoder.cla
plt.title('Confusion Matrix (Unseen Data)')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```




```
errors = unseen_df.copy()

errors["true"] = unseen_labels
errors["pred"] = unseen_predictions

errors = errors[errors["true"] != errors["pred"]]

errors[["rewritten", "true_label", "pred"]].head(20)
```

	rewritten	true_label	pred
4	I am on Zoloft and Focalin and they have chang...	Suicidal	2
5	I have some really bad feelings, but whenever ...	Suicidal	2
31	I do not know how to navigate these feelings, ...	Depression	6
34	My mom made me go to a camp that she knows I g...	Depression	6
37	Since I began seeing a therapist 5 months ago ...	Depression	6
49	Next week I'll be flying for our family vacati...	Stress	3
56	On my way into the hospital Please send good v...	Bipolar	6
57	Husband has simply about blocked me out and re...	Bipolar	2
59	Blegh, tired of this (Rant) (triggering) I sim...	Bipolar	6
62	Anyone else have nothing in common with other ...	Personality disorder	2
64	I am hurting Lately I simply feel like garbage...	Personality disorder	2
65	I cannot cope with my job I work from home as ...	Personality disorder	5
70	I dream that someone would adopt me. I am 23 y...	Personality disorder	2

```
model.eval()
all_probs_unseen = []

with torch.no_grad():
    for batch in unseen_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask
        )
        probabilities = torch.softmax(outputs.logits, dim=1)
        all_probs_unseen.extend(probabilities.cpu().numpy())
```